System Design: Message Brokers

O‘zbek tilida amaliy darslik • AI telefon savdosi misolida

1. Message Broker nima?

Message Broker — tizim komponentlari orasida xabarlarni qabul qiladigan va kerakli iste’molchilarga (consumers) yetkazadigan vositachi. U xizmatlarni bir-biridan bo‘shroq bog‘laydi (loose coupling) va ishlarni asinxron bajarishga yordam beradi.

Misol: AI Sales API suhbatni yakunlagach, lead yaratish, suhbat tarixini saqlash va menejerga xabar yuborish vazifalarini navbatga qo‘yadi. Telefon orqali javob berish yo‘li bu sekin ishlarni kutib qolmaydi.

Producer → Broker / Queue → Consumer (Worker)

Producer xabar yaratadi. Broker uni yo‘naltiradi yoki navbatda saqlaydi. Consumer xabarni olib, vazifani bajaradi. Xabarni saqlash, qayta urinish va yetkazish kafolatlari tanlangan broker hamda konfiguratsiyaga bog‘liq.

2. Nega kerak?

• **Asynchronous processing:** foydalanuvchiga tez javob berib, og‘ir ishni keyin bajarish.

• **Buffering:** so‘rovlar birdan ko‘payganda vazifalarni navbatda ushlab turish.

• **Loose coupling:** producer consumer qayerda va qanday ishlashini bilishi shart emas.

• **Retry:** vaqtinchalik xatoda vazifani qayta bajarish.

• **Load leveling:** worker’lar vazifalarni o‘z imkoniyatiga qarab qayta ishlaydi.

3. Queue va Pub/Sub farqi

Model	Qanday ishlaydi?	Misol
Work Queue	Vazifa odatda bir worker tomonidan bajariladi; bir nechta worker navbatdan vazifalarni bo‘lib oladi.	CRM’ga lead yozish.
Pub/Sub	Bitta event’ga bir nechta subscriber alohida munosabat bildirishi mumkin.	CallCompleted event’idan CRM, analytics va notification xizmatlari foydalanadi.

Eslatma: ayrim brokerlar ikkala usulni ham qo‘llaydi. Queue va Pub/Sub — faqat mahsulot nomi emas, xabarni tarqatish modeli.

4. Mashhur texnologiyalar

RabbitMQ	Task queue va routing uchun qulay; odatda ish navbatlari va qayta urinish mexanizmlarida ishlatiladi.
Apache Kafka	Event streaming, yuqori throughput va event’larni qayta o‘qish (retention) kerak bo‘lganda.
Redis Streams	Redis ekotizimida yengil stream va consumer group imkoniyatlari.
AWS SQS / SNS	AWS’da boshqariladigan queue (SQS) va pub/sub bildirishnomalari (SNS).

Boshlang‘ich tanlov: kichik Python backend va task queue uchun RabbitMQ + Celery ko‘p hollarda tushunarli boshlanish bo‘ladi. Event’larni uzoq muddat saqlash, ko‘p consumer’lar va stream processing kerak bo‘lsa, Kafka ko‘rib chiqiladi. Har doim talabga qarab tanlang.

5. AI Sales tizimi uchun oqim

• Mijoz bilan telefon suhbati tugaydi.

• Backend CallCompleted event’ini yaratadi: call_id, customer_id, transcript reference, timestamp.

• Broker event’ni navbatga yoki topic’ga uzatadi.

• CRM worker lead va suhbat ma’lumotlarini saqlaydi.

• Notification worker menejerga xabar yuboradi.

• Analytics worker suhbat natijasini statistikaga qo‘shadi.

Real-time ogohlantirish: broker odatda telefon suhbatidagi har bir audio bo‘lagining javob yo‘lida bo‘lmasligi kerak. U ko‘proq suhbatdan keyingi vazifalar va kechikishi mumkin bo‘lgan ishlar uchun mos. Real-time audio oqimi alohida past-kechikish arxitekturasini talab qiladi.

6. Amaliy misol: Python + Celery

```
# tasks.py — soddalashtirilgan misol
from celery import Celery

app = Celery(
"sales",
broker="pyamqp://guest@localhost//"
)

@app.task(bind=True, max_retries=3)
def save_lead(self, customer_id, call_id):
try:
# CRM/database ga lead yozish
persist_lead(customer_id, call_id)
except TemporaryError as exc:
raise self.retry(exc=exc, countdown=10)

# API yoki call handler ichida:
save_lead.delay(customer_id, call_id)
```

Bu kod konseptual misol: persist_lead va TemporaryError loyihada yozilishi kerak. Ishlab chiqarishda broker credentials, timeouts, logging, monitoring va idempotency ham kerak.

7. Ishlab chiqarishdagi muhim xavflar

• **Duplicate delivery:** xabar qayta kelishi mumkin. Consumer bir event’ni takroran bajarish zarar keltirmaydigan (idempotent) bo‘lsin.

• **Retry storm:** cheksiz yoki juda tez retry tizimni yanada bosishi mumkin. Exponential backoff va retry limit ishlating.

• **Dead Letter Queue (DLQ):** ko‘p marta xato bergan xabarlarni alohida tekshirish uchun saqlang.

• **Ordering:** xabarlar tartibi har doim kafolatlanmaydi; biznes talabiga qarab partition/key yoki boshqa mexanizm tanlanadi.

• **Durability:** broker restart bo‘lganda xabar yo‘qolmasligi kerak bo‘lsa, persistent message, durable queue va mos acknowledgment sozlamalarini tekshiring.

• **Observability:** queue depth, message age, processing latency, retry count va failed jobs’ni kuzating.

8. Qachon Message Broker kerak emas?

Oddiy CRUD ilovasi, juda kichik trafik va darhol bajarilishi kerak bo‘lgan qisqa operatsiyalar uchun broker ortiqcha murakkablik bo‘lishi mumkin. Avval oddiy arxitekturadan boshlang; asinxron vazifalar, yuklama yoki mustaqil consumer’lar paydo bo‘lganda broker qo‘shing.

9. Mini-quiz

1) AI qo‘ng‘iroq tugagach CRM’ga lead yozish sekin ishlasa, telefon javobini kutkazmaslik uchun qaysi yondashuv foydali?

2) Bitta CallCompleted event’idan CRM, analytics va notification xizmatlari alohida foydalanishi kerak. Qaysi model mos?

3) Consumer xabarni bajarib bo‘ldi, lekin acknowledgment yuborishdan oldin server o‘chdi. Xabar qayta yetkazilishi mumkinmi? Consumer qanday himoyalanishi kerak?

4) Uch marta qayta urinishdan keyin ham xato bo‘layotgan xabarlarni qayerga yo‘naltirish mumkin?

Javoblar (o‘zingni tekshirish uchun): 1) Queue + worker; 2) Pub/Sub yoki event topic; 3) Ha, duplicate ehtimoli bor — idempotency; 4) Dead Letter Queue (DLQ).